

Lecture 23:
Deep Learning
GANs, Self-Supervised Learning
ML/AI for Engineering Applications
ENV 6932

Sara Behdad
Fall 2025

Outline

- **Recall:**
 - Innovations in Convolutional Neural Networks
 - From classification to localization
 - From classification to semantic segmentation
 - RNN (LSTM, GRU)
- **More Advancements:**
 - Unsupervised Learning
 - Self supervised Learning
 - GANs
 - Diffusion Models

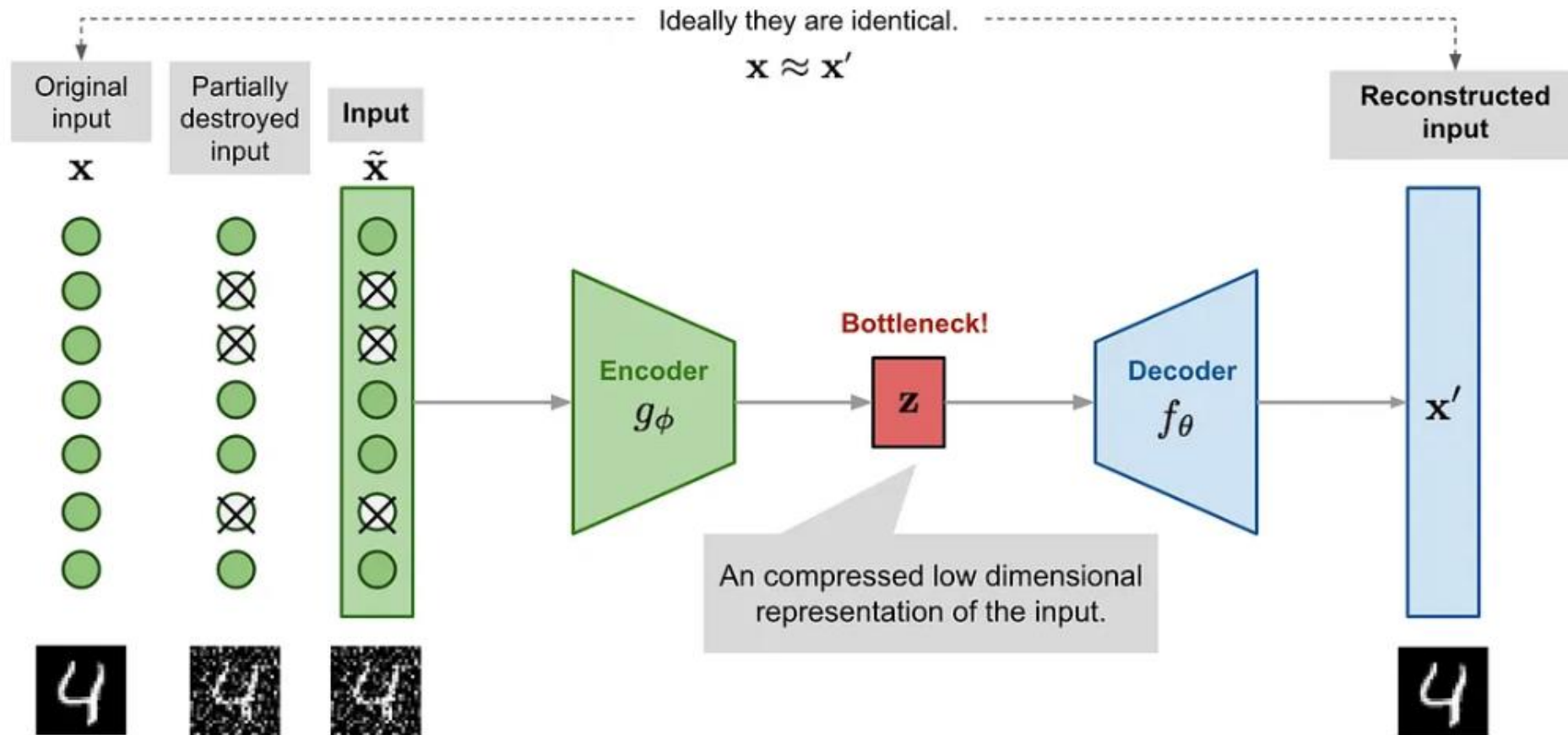
Unsupervised Learning and Representation Learning

- Neural networks learn to compress and reconstruct data.

Autoencoders: Neural Networks for Unsupervised Representation Learning

- We have seen sequence models that predict next words. Now let's look at networks that reconstruct their own input.
- **Why Autoencoders?**
 - Dimensionality Reduction: Learn compact, lower-dimensional representations
 - Feature Learning: Capture meaningful structure without labels
 - Denoising: Reconstruct clean data from noisy input
 - Anomaly Detection: Identify outliers via reconstruction error

Example: Image Denoising Using Autoencoders



Basic Architecture

- Analogy: A smart photocopier that learns to recreate input documents, keeping only the essential details.
- Encoder: compresses input x to latent vector z
- Decoder: reconstructs x' from z
- Training Objective: make x' as close to x as possible

$$z = f_{\theta}(x) \quad (\text{Encoder})$$

$$x' = g_{\phi}(z) \quad (\text{Decoder})$$

$$L = \|x - x'\|^2 \quad (\text{Reconstruction loss})$$

Variational Autoencoders (VAE): Probabilistic Representation Learning

- We saw how classical autoencoders learn to reconstruct data. But what if we want them to generate new, realistic samples, not just copy inputs?
- Instead of one latent code per input, the network learns a distribution.
- Adds probabilistic latent variables:

$$z \sim \mathcal{N}(\mu, \sigma^2)$$

- Loss:

$$L = \|x - x'\|^2 + D_{KL}(q(z|x) \| p(z))$$

Variational Autoencoders (VAE): Probabilistic Representation Learning

- We saw how classical autoencoders learn to reconstruct data. But what if we want them to generate new, realistic samples, not just copy inputs?
- Instead of one latent code per input, the network learns a distribution.
- In a standard autoencoder:

$$x \xrightarrow{\text{Encoder}} z \xrightarrow{\text{Decoder}} \hat{x}$$

- z is a fixed vector per input.
- In a VAE:

$$x \xrightarrow{\text{Encoder}} (\mu, \sigma) \xrightarrow{\text{Sampling}} z \xrightarrow{\text{Decoder}} \hat{x}$$

The encoder outputs a distribution (mean and variance) over possible latent representations.

Kullback-Leibler Divergence

- In a Variational Autoencoder:
 - $q_\phi(z | x)$ the encoder's learned distribution over latent variables (posterior).
 - $p(z)$ the prior distribution, usually $\mathcal{N}(0, I)$.
- The **KL divergence term**: $D_{KL} \left(q_\phi(z | x) \parallel p(z) \right)$
- encourages the learned latent distribution $q(z | x)$ to stay close to the standard normal $\mathcal{N}(0, I)$.
- **Recall**: DKL measures how different one probability distribution P is from another distribution Q .

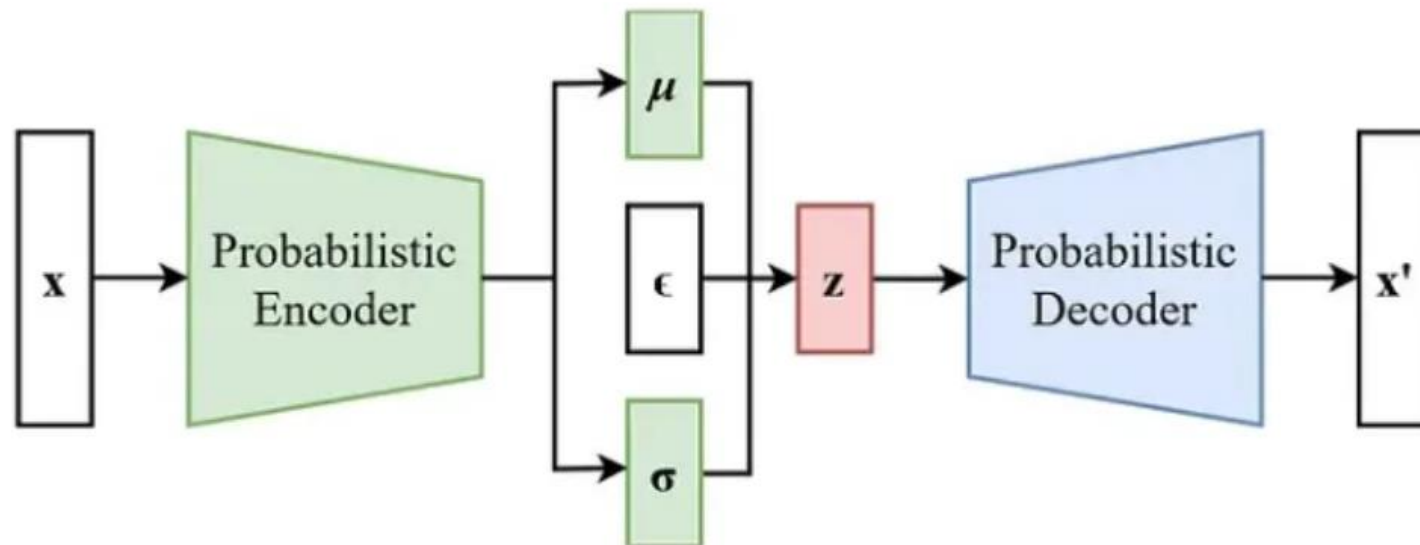
$$D_{KL}(P \parallel Q) = \int P(x) \log \frac{P(x)}{Q(x)} dx$$

Kullback-Leibler Divergence

- The **KL divergence term**: $D_{KL} \left(q_{\phi}(z | x) \parallel p(z) \right)$
 - Without this term: The encoder might learn arbitrary, scattered distributions for each input.
 - With KL divergence: Latent representations are pushed toward a shared Gaussian structure.

VAE

- VAE is used to learn a compressed representation of data while also being able to generate new, synthetic data points.
- VAEs incorporate probabilistic inference, helping them generate new samples by sampling from a learned latent distribution.



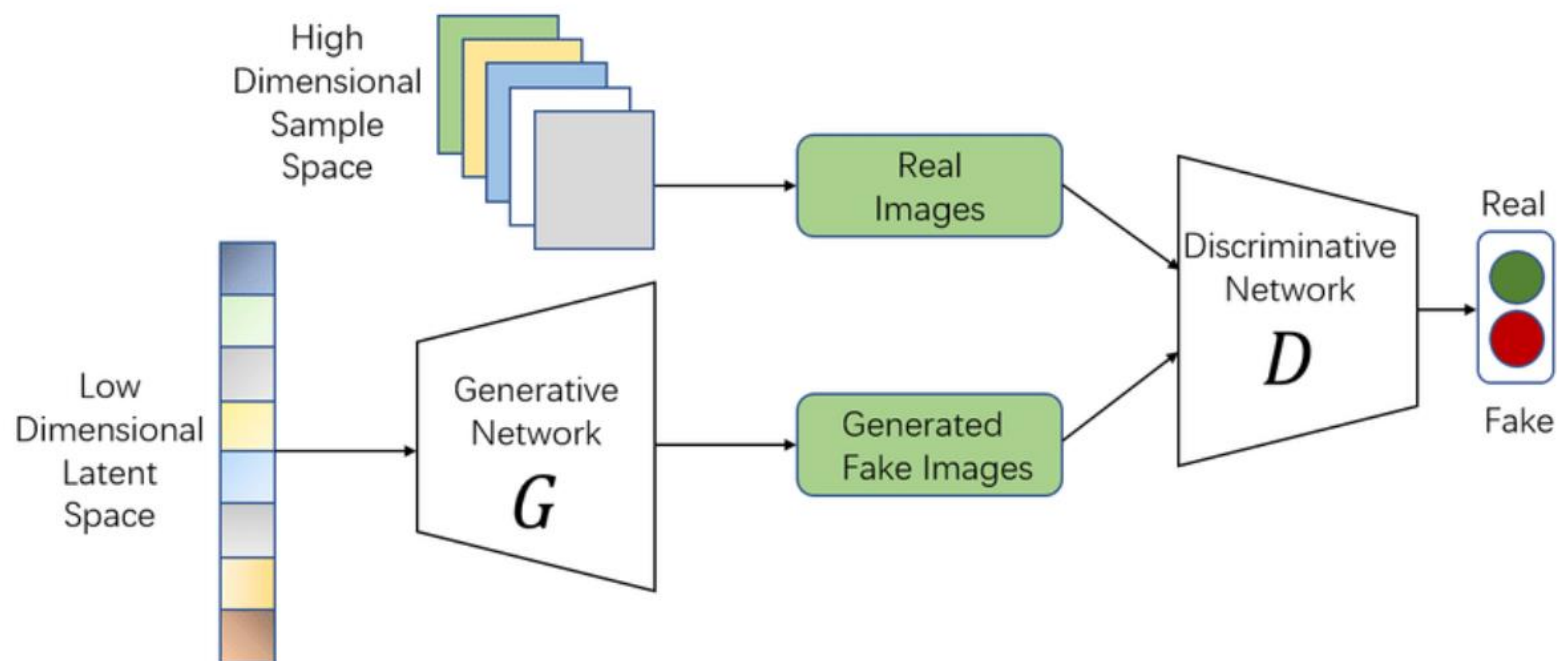
Generative Adversarial Networks (GANs)

- “The most interesting idea in the last 10 years in machine learning.”
Yann LeCun
- GAN: proposed by Ian Goodfellow et al., 2014.
- A GAN consists of two neural networks trained against each other:
 - **Generator (G):** produces fake samples from random noise
 - **Discriminator (D):** classifies samples as real or fake

The Generator learns to fool the Discriminator, while the Discriminator learns to detect fakes.

Motivation

Model	Idea	Limitation
Autoencoder / VAE	Learn to reconstruct input	Outputs can look blurry
GAN	Learn to generate data that looks real	More difficult to train, but produces sharper outputs



Applications

Area	Example
Computer Vision	Image synthesis, super-resolution, inpainting
Healthcare	Synthetic medical data, privacy-preserving samples
Art and Design	Style transfer, creative generation
Manufacturing and Sustainability	Synthetic defect data, product reconstruction
Autonomous Systems	Simulated rare events for training
Data Augmentation	Generate diverse training data when real data is scarce

The GAN Loss Function

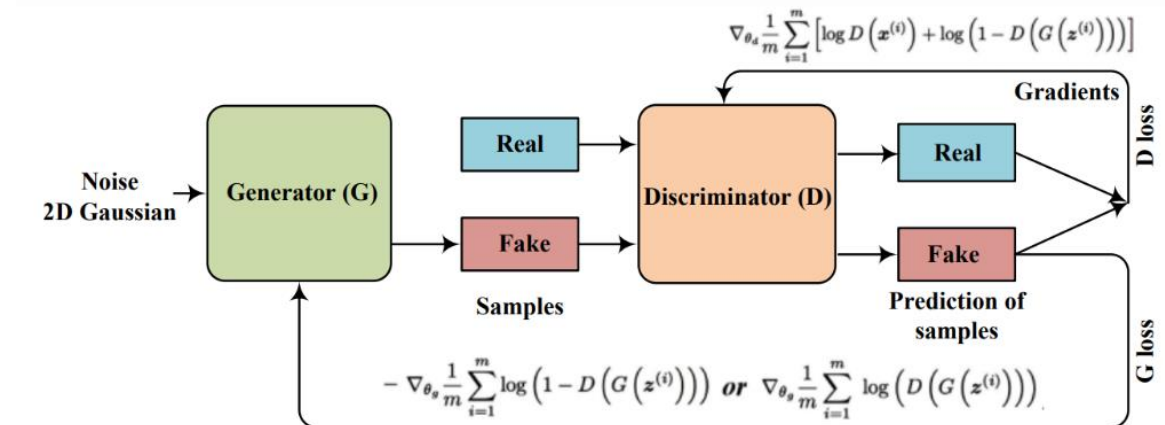
- The original GAN (Goodfellow et al., 2014) defines a minimax objective:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

- G: The generator network that takes random noise (z) and produces synthetic (fake) data samples G(z).
- D: The discriminator network that takes an input sample and outputs a number between 0 and 1, representing how “real” the sample looks.
- X: A real data sample from the dataset (for example, a true image of a cat).
- z : A noise vector drawn from a simple prior distribution (such as a Gaussian).
- $p_{data}(x)$: The true data distribution, the probability distribution of real samples in the dataset.
- $p_z(z)$: The noise (latent) distribution used to generate input noise for the generator.
- $E[\cdot]$: The average value of the expression over many samples.
- $D(x)$: The output of the discriminator for input x, interpreted as the probability that x is real (not generated).

The GAN Loss Function

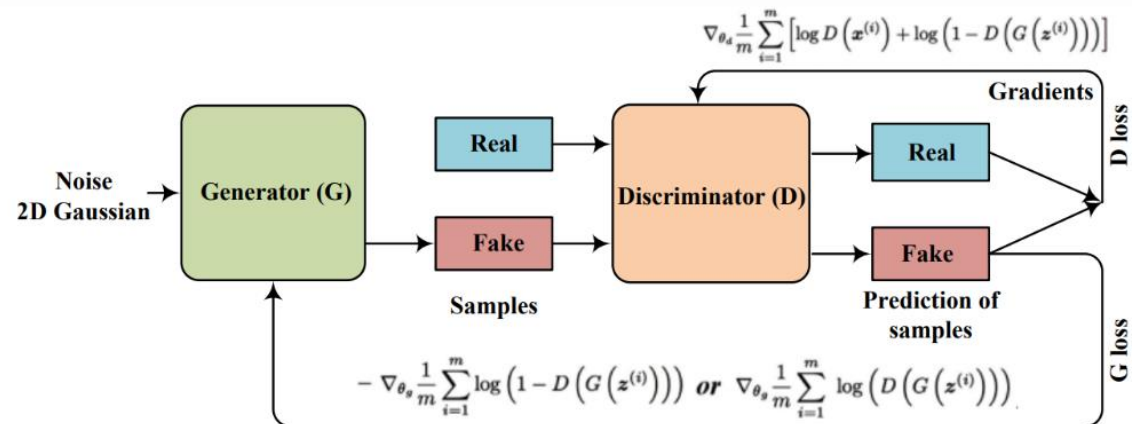
- **Term 1:** $\mathbb{E}_{x \sim p_{data}(x)} [\log D(x)]$
- Feed real data into D.
- D(x) should be close to **1** (real).
- The log is used because it penalizes confident wrong predictions strongly (standard for probabilistic classifiers).
- D wants this term to be large, meaning it correctly recognizes real data.



The GAN Loss Function

Term 2: $\mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$

- Feed fake data $G(z)$ (generated from noise) into D.
- $D(G(z))$ should be close to **0** (fake).
- Again, log penalizes confident mistakes.
- D wants this term large, meaning it confidently rejects fakes.



The Minimax Game

- The **Discriminator (D)** tries to **maximize** the total objective:

$$V(D, G) = \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

- i.e., get $D(x) \approx 1$ and $D(G(z)) \approx 0$
- The **Generator (G)** tries to **minimize** that same objective:

$$\min_G V(D, G)$$

make $D(G(z))$ as close to 0 as possible (fool D).

- In short:
- D wants: “**Real =1, Fake =0.**”
- G wants: “**Fake =1.**”

Self-Supervised Learning (SSL)

- Traditional supervised learning needs large labeled datasets: costly and domain-specific.
- Unsupervised models (Autoencoders, VAEs, GANs) learn to model or generate data, but not always good at learning transferable features.
- Self-Supervised Learning solves this:
 - Uses unlabeled data,
 - Creates its own supervision from the structure of data,
 - Learns representations useful for many downstream tasks.

Intuition Behind SSL

- Unsupervised learning (VAE, GAN):
- The model tries to understand the data itself, how to reconstruct or generate it.

“If I can produce realistic data, I must have understood the structure of the data distribution.”
- SSL: The model learns by solving a surrogate (pretext) task designed to teach meaningful representations.

“If I can predict missing parts or identify similar views, I must have learned what is important in the data.”

What Is Self-Supervised Learning?

- Generate pseudo-labels from the data itself (e.g., predict missing parts, align similar views).
- **Objective:**

$$\min_{\theta} L_{\text{pretext}}(f_{\theta}(x))$$

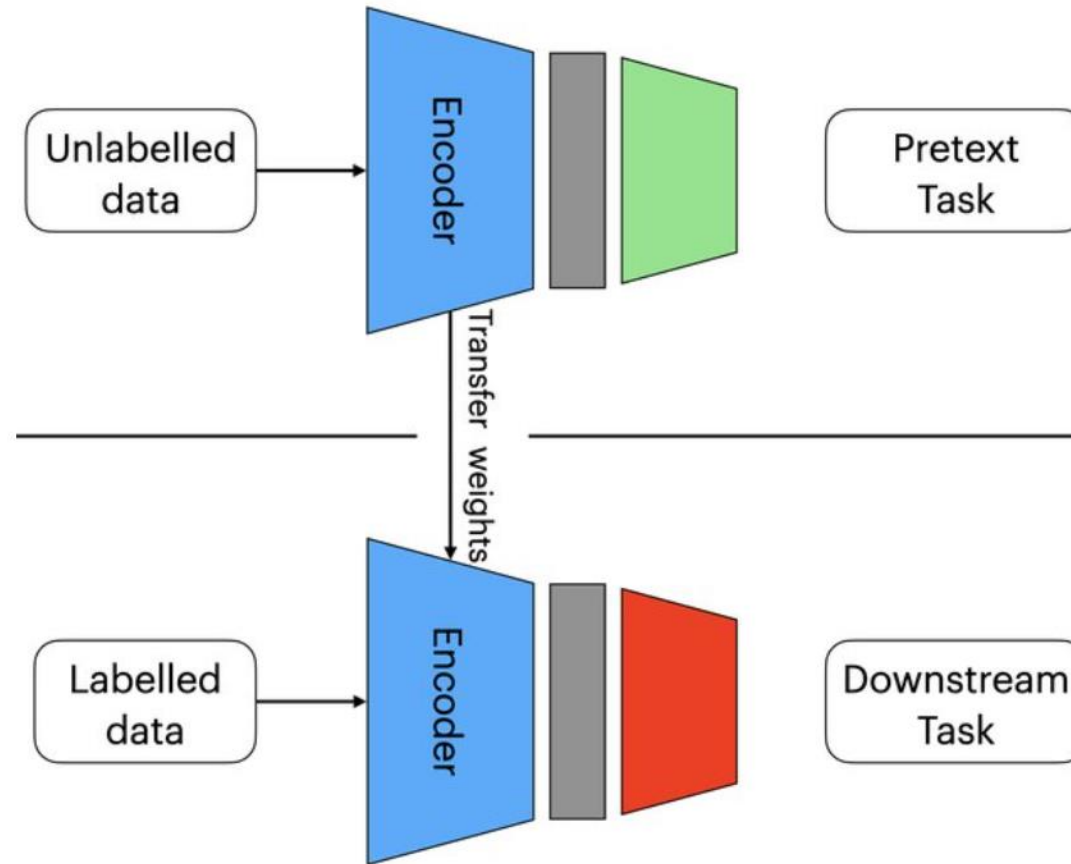
where f_{θ} is an encoder that learns reusable features.

- **Output:** Encoder representations $h=f_{\theta}(x)$ transferable to classification, detection, etc.

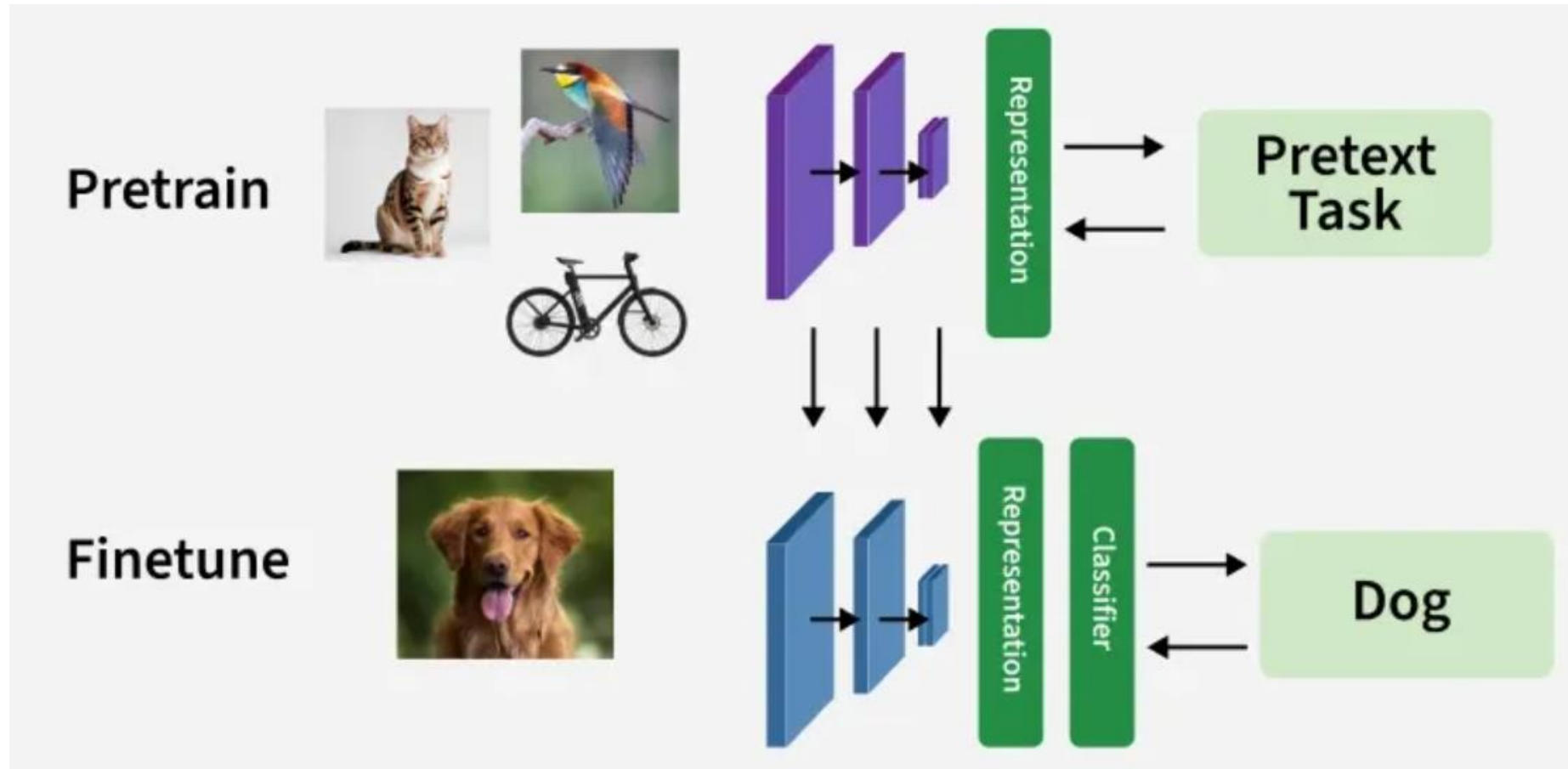
How It Differs from Unsupervised Generative Models

Aspect	Unsupervised (VAE, GAN)	Self-Supervised (SSL)
Goal	Model or generate data $p(x)$	Learn representations $f(x)$
Supervision	None (learn from reconstruction or adversarial signal)	Automatically generated pseudo-labels
Output	Synthetic data (image, signal)	Embedding or feature vector
Example Tasks	Reconstruct, generate	<u>Predict, align, contrast</u>
Use Case	Data synthesis	<u>Transferable features</u>

Simple schematic representation of the SSL paradigm



Simple schematic representation of the SSL paradigm



Overview of SSL Architectures

- **Two main families:**
 - **Contrastive Learning:** learn by comparing examples
 - **Predictive / Masked Learning:** learn by reconstructing missing parts
- **Other directions:**
 - **Cross-Modal SSL:** learn from paired modalities (e.g., text and image)
 - **Generative and SSL Hybrids:** combine reconstruction and discrimination (e.g., BEiT)

1. Contrastive Learning

- **Objective:** Learn an encoder that produces *similar embeddings* for related samples and *different embeddings* for unrelated ones.
- **Process:**
 - Create two *views* of the same data (e.g., two augmentations of an image).
 - Pass them through the same encoder f_θ .
 - Learn representations h_1, h_2 such that:
 $\text{sim}(h_1, h_2)$ is high, and $\text{sim}(h_1, h_k)$ (for other samples) is low.
- **Typical Loss:**
InfoNCE (Contrastive Loss)
$$L = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_k \exp(\text{sim}(z_i, z_k)/\tau)}$$
- where z_i, z_j are positive pairs and τ controls sharpness.

1. Contrastive Learning

Example:

Input	Encoder Output (Embedding)	Meaning
Image of a cat	[0.2, 0.8, 0.1, 0.9, ...]	Cat-like features
Image of a dog	[0.3, 0.9, 0.2, 0.8, ...]	Dog-like features
Image of a car	[0.9, 0.1, 0.7, 0.1, ...]	Car-like features

After training, embeddings of similar objects (cats, dogs) will be near each other in the embedding space, while unrelated ones (cars) will be far apart.

Applications of Contrastive SSL

- Vision: Pretraining for detection, classification, segmentation (SimCLR, MoCo)
- NLP: Sentence similarity and retrieval (SimCSE)
- Audio: Speaker verification, sound event detection
- Multimodal: CLIP for text-image zero-shot recognition
- Industrial: Similarity-based defect detection, condition monitoring

2. Predictive/Masked Learning

- Learn to understand the structure of data by *predicting missing or hidden parts*.
- **Process:**
 - Randomly hide (mask) portions of input data.
 - Train encoder to predict or reconstruct those parts.

- **Objective:**

$$L = \|x_{\text{reconstructed}} - x_{\text{original}}\|^2 \text{ or } L = - \sum_{t \in M} \log P(w_t | w_{\setminus M})$$

- **Outcome:**
Encoders learn contextual and global representations.

Cross-Entropy Loss in Self-Supervised Language Models (BERT)

- Measure how well the model predicts the correct masked word using surrounding context.
- Example: The cat sat on the [MASK].

$$L = - \sum_{t \in M} \log P_{\theta}(w_t \mid w_{\setminus M})$$

Notation

- M : Set of masked token positions (e.g., the words replaced by [MASK]).
- w_t : The *true* word that was masked (ground truth).
- $w_{\setminus M}$: The *visible* words (context).
- $P_{\theta}(w_t \mid w_{\setminus M})$: Model's *predicted probability* of the correct word given its context.
- θ : Model parameters (Transformer weights).

Applications of Predictive SSL

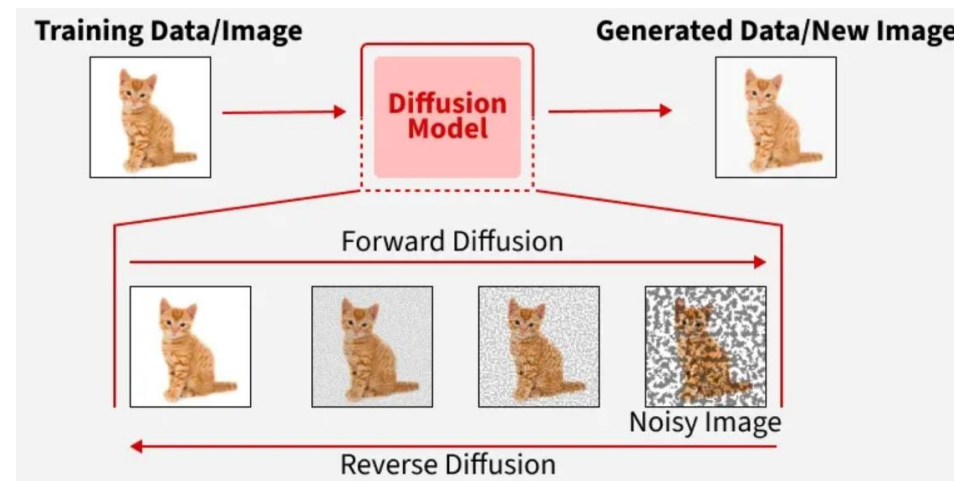
- Language: BERT, GPT pretraining for NLP understanding and generation.
- Vision: MAE, BEiT for image recognition and scene understanding.
- Audio: Wav2Vec 2.0 learns from raw speech for ASR.
- Engineering/Sustainability: Masked sensor prediction, anomaly detection, equipment fault diagnosis, or missing data reconstruction.

Diffusion Models: The Next Generation of Generative AI

- **Motivation: Why Diffusion Models?**
- GANs are powerful but unstable: training can collapse or produce artifacts.
- VAEs produce blurry results due to strong assumptions about distributions.
- Diffusion models:
 - Simple to train (no adversarial setup)
 - Stable, high-quality samples
 - State-of-the-art in image, video, and text-to-image generation.

Diffusion Models

- Diffusion models learn to generate data by gradually reversing a noising process.
1. **Forward process (diffusion):** gradually add random noise to real data, destroy structure.
 2. **Reverse process (denoising):** train a neural network to remove noise step by step, recreate data from pure noise.

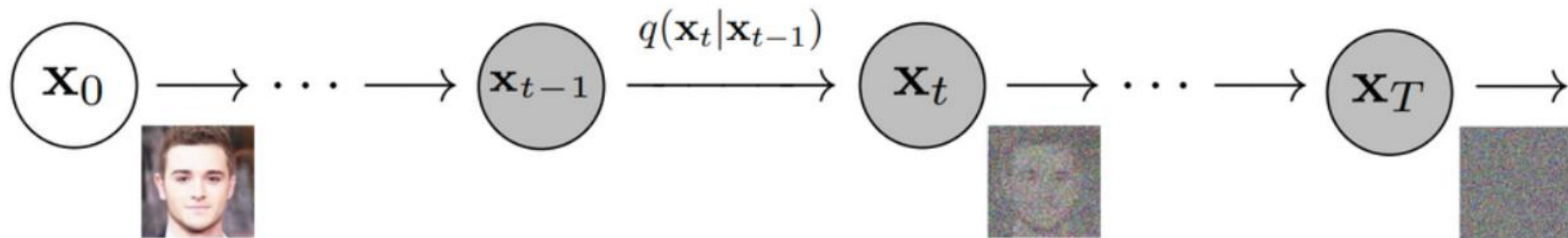


Forward Diffusion Process

- We start with a clean image x_0 .
- At each step t , we add a small amount of Gaussian noise:

$$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t} \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I)$$

- After many steps, x_T becomes nearly pure noise.
- α_t :controls how much noise is added per step (called *variance schedule*).



Forward Diffusion P

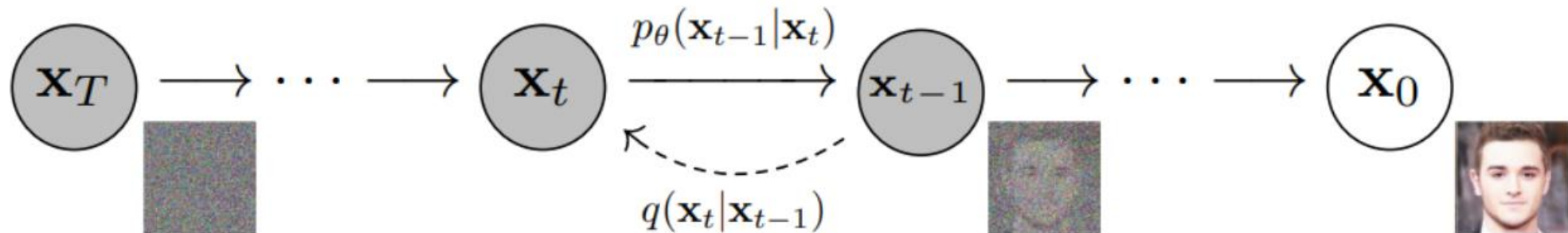
Reverse (Denoising) Process

- We train a model $\epsilon_{\theta}(x_t, t)$ to predict the added noise.
- The model learns **how to denoise** x_t step by step:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$$

where $z \sim \mathcal{N}(0, I)$.

- **Logic:** If we can predict and remove the noise accurately, we can recover a clean image.



Reverse Diffusion Process

Training Objective

- The diffusion model minimizes a simple mean squared error (MSE) between:
 - the *true noise* ϵ_t used in forward process
 - the *predicted noise* $\epsilon_\theta(x_t, t)$:

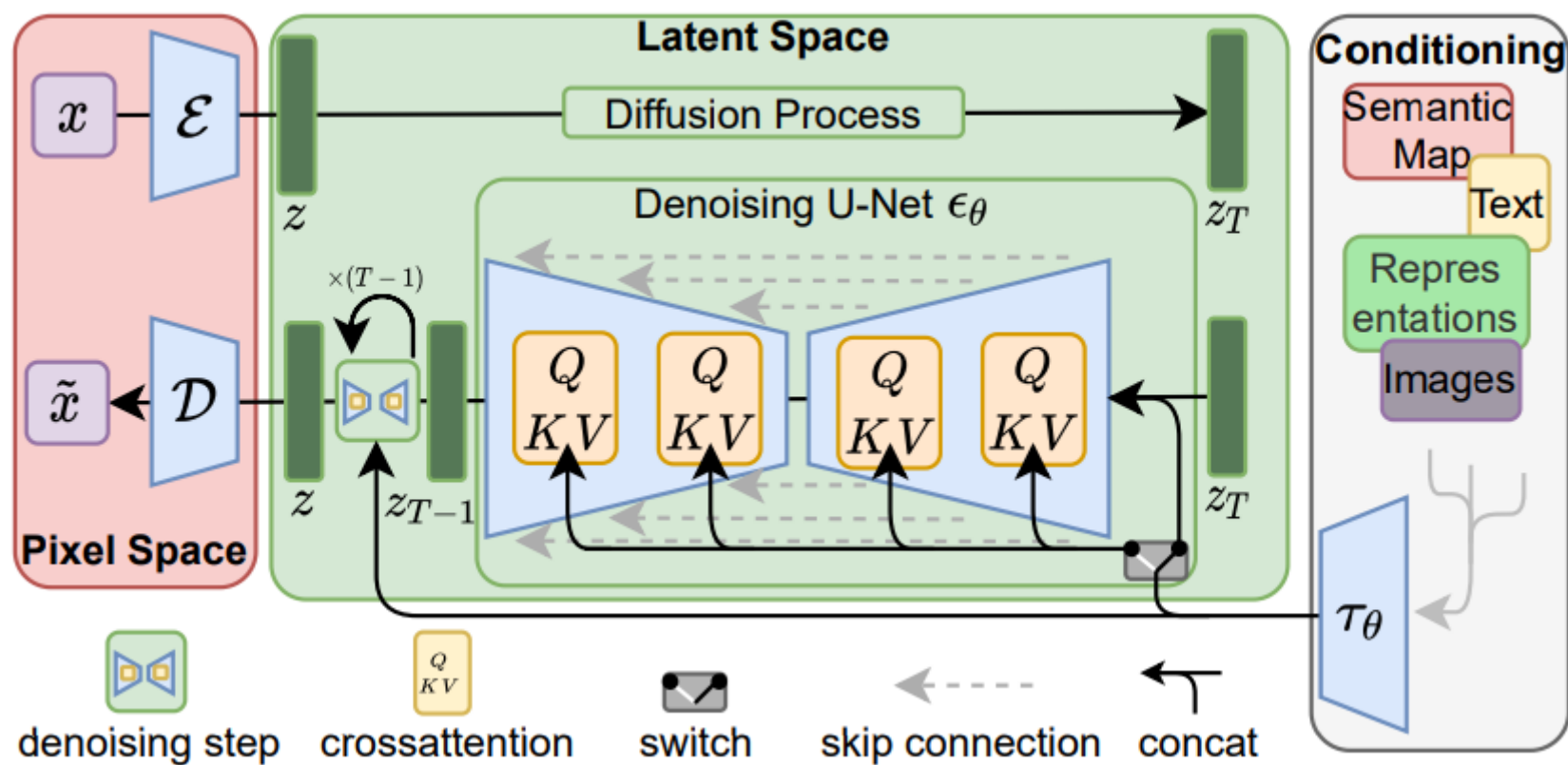
$$L_{\text{simple}} = \mathbb{E}_{x_0, t, \epsilon_t} \left[\|\epsilon_t - \epsilon_\theta(x_t, t)\|_2^2 \right]$$

- This is equivalent to maximizing likelihood under a Gaussian generative process.
- **Key advantage:** stable, non-adversarial training.

General Process

- Take a real image, add random noise gradually until it becomes pure noise.
- Train a network to predict the noise at each step.
- If the network can correctly remove noise, it learns the structure of real data.
- At generation time, start with random noise, denoise step by step, get a new realistic image.
- Note: Most diffusion models use U-Net architecture: Encoder-decoder with skip connections.

Diffusion Architecture

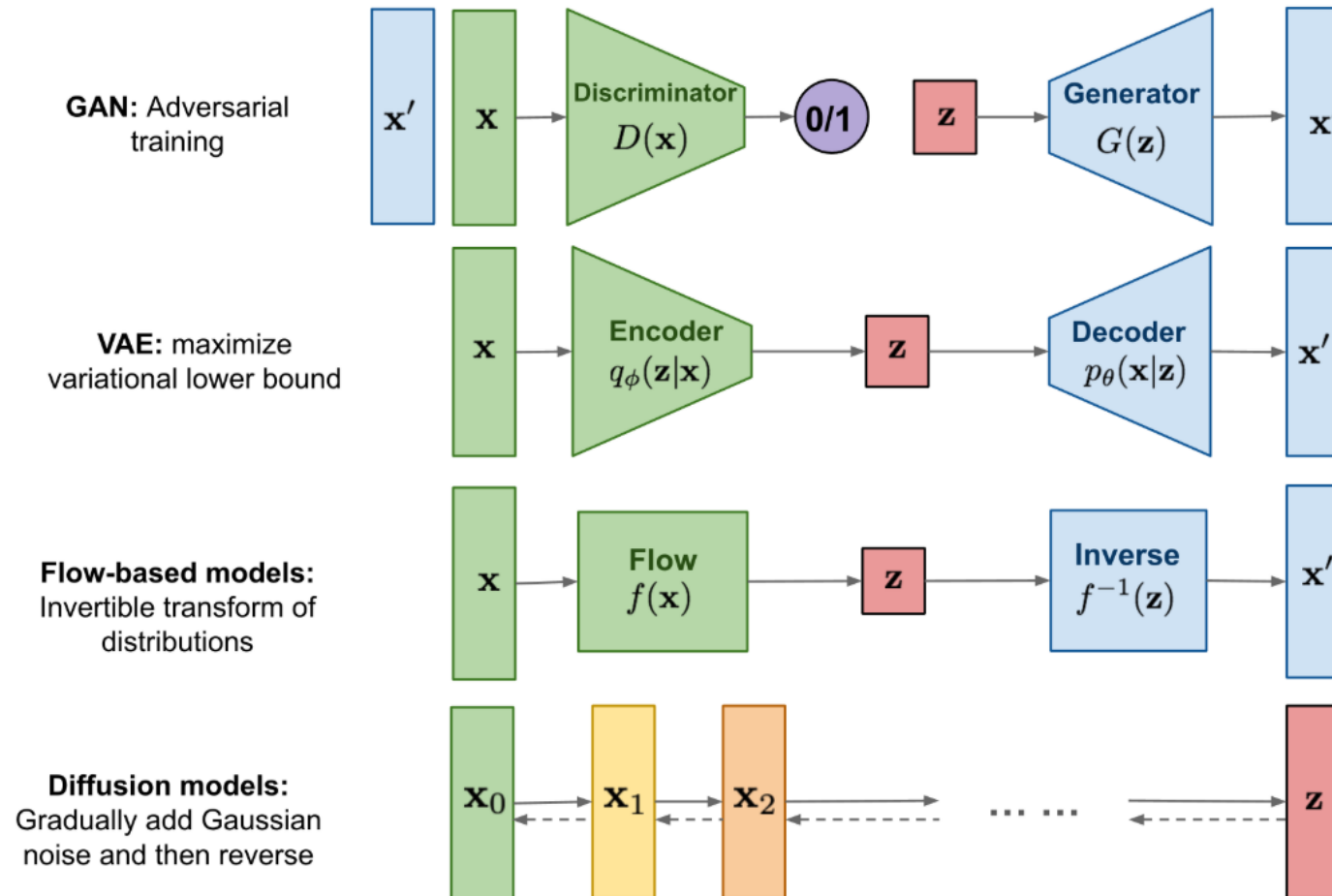


Conditioning and Control

- Diffusion models can be conditioned on additional information:
- Conditional diffusion = guided denoising based on external input.

Condition Type	Example	Application
Text	A cat sitting on grass	DALL·E, Stable Diffusion
Class label	digit 3	Class-conditional diffusion
Sketch / image	Inpainting, editing	ControlNet, image restoration
Audio	Speech synthesis	WaveGrad, DiffWave

Overview of different types of generative models



Comparison with Other Generative Models

Model	Key Mechanism	Strengths	Weaknesses
VAE	Encode-decode, latent sampling	Probabilistic, stable	Blurry results
GAN	Adversarial generator-discriminator	Sharp images	Unstable, mode collapse
Diffusion	Denoising via noise prediction	Stable, high quality	Slow sampling
Flow-based	Invertible transformations	Exact likelihood	Computationally heavy